

Selection Sort

Scan the entire unsorted suffix, remember its minimum index, and swap that minimum into the next final position.

This complete offline chapter connects ideas, diagrams, exact traces, commented Java and exam practice. Switch themes for reading; printing always uses the light presentation and includes every required trace and answer.

GUIDE 2026-10-07.1 · STANDALONE HTML AND PDF SHARE THE SAME CONTENT

1. The array we will sort

Arrays use zero-based positions. Sorting changes which values occupy those positions. The target is ascending order; record tags would move with their values.



Initial input [4,2,5,1,3]. Five fixed index positions 0..4.

We will distinguish comparisons, swaps, shifts, temporary writes and copy-back writes. They are different units.

2. Definition and plain language

Selection sort partitions an array into a sorted prefix and unsorted suffix. Each pass selects the smallest suffix value and exchanges it with the first suffix position.

Scan the whole suffix



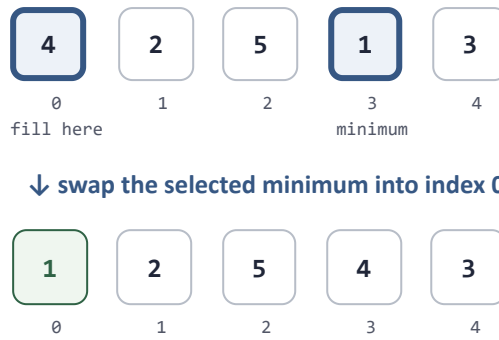
Remember best index



Fill next final position

The algorithm has a distinct decision and movement rule; each completed pass grows its known-sorted region.

Find the smallest remaining card before you move anything. Put it in the next empty final position. Repeat with the cards still waiting.



A concrete first pass: highlighted positions explain the choice; the green region records the completed result.

The scan proves which value belongs at index i ; one swap puts it there. A new smaller candidate updates an index during the scan rather than causing an immediate swap.

3. Vocabulary and the central invariant

Term	Meaning in this chapter
i	The final position being filled.
min	Index of the best candidate seen so far, not the candidate value.
j	Index currently being scanned.
Sorted prefix	Positions $0..i-1$ already contain the smallest i values.
Selection	Finding a minimum before one final swap.

Invariant: Before pass i , positions $0..i-1$ contain the i smallest values in final order. During the scan, min names the smallest value seen from i through j .

4. Step through the complete execution

Predict the next decision before advancing. Next updates both the scene and the visibly marked trace row. Print includes the completed states and full table.

One comparison or write at a time

▼ Full trace table - select a row to jump to its scene

Step	Operation	Reason	Shared array
1	Pass 1, scan index 1	$2 < 4 \rightarrow \text{true}$; best index becomes 1	[4, 2, 5, 1, 3]

Step	Operation	Reason	Shared array
2	Pass 1, scan index 2	$5 < 2 \rightarrow \text{false}$; best stays 1	[4, 2, 5, 1, 3]
3	Pass 1, scan index 3	$1 < 2 \rightarrow \text{true}$; best index becomes 3	[4, 2, 5, 1, 3]
4	Pass 1, scan index 4	$3 < 1 \rightarrow \text{false}$; best stays 3	[4, 2, 5, 1, 3]
5	Pass 1, fill position 0	Scan is complete: swap indexes 0 and 3.	[1, 2, 5, 4, 3]
6	Pass 2, scan index 2	$5 < 2 \rightarrow \text{false}$; best stays 1	[1, 2, 5, 4, 3]
7	Pass 2, scan index 3	$4 < 2 \rightarrow \text{false}$; best stays 1	[1, 2, 5, 4, 3]
8	Pass 2, scan index 4	$3 < 2 \rightarrow \text{false}$; best stays 1	[1, 2, 5, 4, 3]
9	Pass 2, fill position 1	Minimum is already in position; no swap.	[1, 2, 5, 4, 3]
10	Pass 3, scan index 3	$4 < 5 \rightarrow \text{true}$; best index becomes 3	[1, 2, 5, 4, 3]
11	Pass 3, scan index 4	$3 < 4 \rightarrow \text{true}$; best index becomes 4	[1, 2, 5, 4, 3]
12	Pass 3, fill position 2	Scan is complete: swap indexes 2 and 4.	[1, 2, 3, 4, 5]
13	Pass 4, scan index 4	$5 < 4 \rightarrow \text{false}$; best stays 3	[1, 2, 3, 4, 5]
14	Pass 4, fill position 3	Minimum is already in position; no swap.	[1, 2, 3, 4, 5]

Pass 1, fill position 0

12

1

2

5

4

3

0

1

2

3

4

Indexes remain fixed. Highlighted cells are the current comparison or write; green prefix/suffix is known sorted.

Scan is complete: swap indexes 0 and 3.

Pass 2, fill position 1

12

1

2

5

4

3

0

1

2

3

4

Indexes remain fixed. Highlighted cells are the current comparison or write; green prefix/suffix is known sorted.

Minimum is already in position; no swap.

Pass 3, fill position 2



Indexes remain fixed. Highlighted cells are the current comparison or write; green prefix/suffix is known sorted.

Scan is complete: swap indexes 2 and 4.

Pass 4, fill position 3



Indexes remain fixed. Highlighted cells are the current comparison or write; green prefix/suffix is known sorted.

Minimum is already in position; no swap.

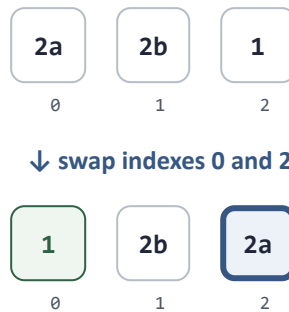
Final result: [1, 2, 3, 4, 5].

5. Exam-style completed-pass / return table

One row per outer-loop pass. These are the completed states to reproduce when an exam asks for the array after each pass.

Pass	Array before	Evaluated comparisons / decisions	Array after	Key comparisons	Swaps
1	[4,2,5,1,3]	$2 < 4 \rightarrow \text{true}$; best index becomes 1; $5 < 2 \rightarrow \text{false}$; best stays 1; $1 < 2 \rightarrow \text{true}$; best index becomes 3; $3 < 1 \rightarrow \text{false}$; best stays 3; Scan is complete: swap indexes 0 and 3.	[1,2,5,4,3]	4	1
2	[1,2,5,4,3]	$5 < 2 \rightarrow \text{false}$; best stays 1; $4 < 2 \rightarrow \text{false}$; best stays 1; $3 < 2 \rightarrow \text{false}$; best stays 1; Minimum is already in position; no swap.	[1,2,5,4,3]	3	0
3	[1,2,5,4,3]	$4 < 5 \rightarrow \text{true}$; best index becomes 3; $3 < 4 \rightarrow \text{true}$; best index becomes 4; Scan is complete: swap indexes 2 and 4.	[1,2,3,4,5]	2	1
4	[1,2,3,4,5]	$5 < 4 \rightarrow \text{false}$; best stays 3; Minimum is already in position; no swap.	[1,2,3,4,5]	1	0

6. What actually moves, and why



Equal keys 2a and 2b reverse relative order. Ordinary swapping selection sort is not stable.

The tags distinguish records whose comparison key is the same. A stable algorithm must preserve those tags in their original order.

7. Fully commented Java implementation

JAVA

```
public static void selectionSort(int[] arr) {
    // Fill each final position from left to right.
    for (int i = 0; i < arr.length - 1; i++) {
        int min = i; // Store an INDEX, not a value.
        // Scan the complete unsorted suffix before swapping anything.
        for (int j = i + 1; j < arr.length; j++) {
            if (arr[j] < arr[min]) min = j;
        }
        // At most one swap per pass; skip a redundant self-swap.
        if (min != i) {
            int temp = arr[i]; arr[i] = arr[min]; arr[min] = temp;
        }
    }
}
```

Write it from scratch

- Choose the next final position.
- Initialize the minimum index to it.
- Scan the complete suffix.
- Remember better candidate indexes.
- Perform at most one final swap.

8. Recursion, calls and returns

JAVA

```
public static void selectionSortRec(int[] arr, int i) {
    // Zero or one position remains to fill.
    if (i >= arr.length - 1) return;
    int min = i;
    for (int j = i + 1; j < arr.length; j++) {
        if (arr[j] < arr[min]) min = j;
    }
    if (min != i) {
        int temp = arr[i]; arr[i] = arr[min]; arr[min] = temp;
    }
    // The completed prefix grows; the remaining suffix shrinks.
    selectionSortRec(arr, i + 1);
    // First call: selectionSortRec(arr, 0).
}
```

CALL STACK - NEWEST FRAME AT THE TOP

selectionSortRec(i=1)

Scan suffix 1..4 and fill index 1

selectionSortRec(i=0)

Waiting: Child fills remaining positions 1..4

Unlike iterative auxiliary space, suspended recursive frames require $O(n)$ stack space in the worst case.

9. Complete runnable program

Save as SelectionDemo.java. The code includes the method(s) and a main entry point.

JAVA

```
import java.util.Arrays;

public class SelectionDemo {
    public static void selectionSort(int[] arr) {
        // Fill each final position from left to right.
        for (int i = 0; i < arr.length - 1; i++) {
            int min = i; // Store an INDEX, not a value.
            // Scan the complete unsorted suffix before swapping anything.
            for (int j = i + 1; j < arr.length; j++) {
                if (arr[j] < arr[min]) min = j;
            }
            // At most one swap per pass; skip a redundant self-swap.
            if (min != i) {
                int temp = arr[i]; arr[i] = arr[min]; arr[min] = temp;
            }
        }
    }
}
```

```

    }
}

public static void main(String[] args) {
    int[] values = {4, 2, 5, 1, 3};
    selectionSort(values);
    System.out.println(Arrays.toString(values));
}
}

```

EXPECTED CONSOLE OUTPUT

[1, 2, 3, 4, 5]

10. Descending order: change the decision

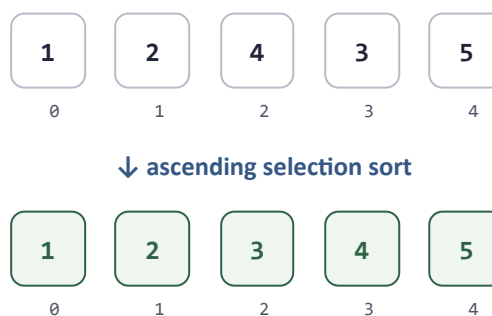
Select the maximum instead: change `arr[j] < arr[min]` to `arr[j] > arr[min]`. The prefix then grows in descending order.

Keep the bounds and movement rules consistent. Reversing a final array is a different procedure and does not reproduce the descending trace of this implementation.

11. Five worked examples

Each case has input and result diagrams, evaluated decisions or merge inputs, exact counts, and a final state.

1. Nearly sorted



The trace below connects the input diagram to the final result.

Pass	Before	Evaluated conditions	After	Key comparisons
1	[1,2,4,3,5]	$2 < 1 \rightarrow \text{false}$; best stays 0; $4 < 1 \rightarrow \text{false}$; best stays 0; $3 < 1 \rightarrow \text{false}$; best stays 0; $5 < 1 \rightarrow \text{false}$; best stays 0; Minimum is already in position; no swap.	[1,2,4,3,5]	4
2	[1,2,4,3,5]	$4 < 2 \rightarrow \text{false}$; best stays 1; $3 < 2 \rightarrow \text{false}$; best stays 1; $5 < 2 \rightarrow \text{false}$; best stays 1; Minimum is already in position; no swap.	[1,2,4,3,5]	3
3	[1,2,4,3,5]	$3 < 4 \rightarrow \text{true}$; best index becomes 3; $5 < 3 \rightarrow \text{false}$; best stays 3; Scan is complete: swap indexes 2 and 3.	[1,2,3,4,5]	2
4	[1,2,3,4,5]	$5 < 4 \rightarrow \text{false}$; best stays 3; Minimum is already in position; no swap.	[1,2,3,4,5]	1

10 key comparisons; 1 swaps.

2. Reverse order

5

4

3

2

1

0

1

2

3

4

↓ ascending selection sort

1

2

3

4

5

0

1

2

3

4

The trace below connects the input diagram to the final result.

Pass	Before	Evaluated conditions	After	Key comparisons
1	[5,4,3,2,1]	$4 < 5 \rightarrow \text{true}$; best index becomes 1; $3 < 4 \rightarrow \text{true}$; best index becomes 2; $2 < 3 \rightarrow \text{true}$; best index becomes 3; $1 < 2 \rightarrow \text{true}$; best index becomes 4; Scan is complete: swap indexes 0 and 4.	[1,4,3,2,5]	4
2	[1,4,3,2,5]	$3 < 4 \rightarrow \text{true}$; best index becomes 2; $2 < 3 \rightarrow \text{true}$; best index becomes 3; $5 < 2 \rightarrow \text{false}$; best stays 3; Scan is complete: swap indexes 1 and 3.	[1,2,3,4,5]	3
3	[1,2,3,4,5]	$4 < 3 \rightarrow \text{false}$; best stays 2; $5 < 3 \rightarrow \text{false}$; best stays 2; Minimum is already in position; no swap.	[1,2,3,4,5]	2
4	[1,2,3,4,5]	$5 < 4 \rightarrow \text{false}$; best stays 3; Minimum is already in position; no swap.	[1,2,3,4,5]	1

10 key comparisons; 2 swaps.

3. Duplicate values

2

2

1

3

0123

↓ ascending selection sort

1

2

2

3

0123

The trace below connects the input diagram to the final result.

Pass	Before	Evaluated conditions	After	Key comparisons
1	[2,2,1,3]	$2 < 2 \rightarrow \text{false}$; best stays 0; $1 < 2 \rightarrow \text{true}$; best index becomes 2; $3 < 1 \rightarrow \text{false}$; best stays 2; Scan is complete: swap indexes 0 and 2.	[1,2,2,3]	3
2	[1,2,2,3]	$2 < 2 \rightarrow \text{false}$; best stays 1; $3 < 2 \rightarrow \text{false}$; best stays 1; Minimum is already in position; no swap.	[1,2,2,3]	2
3	[1,2,2,3]	$3 < 2 \rightarrow \text{false}$; best stays 2; Minimum is already in position; no swap.	[1,2,2,3]	1

6 key comparisons; 1 swaps.

4. Uneven length

38

27

43

3

9

01234

↓ ascending selection sort

3

9

27

38

43

01234

The trace below connects the input diagram to the final result.

Pass	Before	Evaluated conditions	After	Key comparisons
1	[38,27,43,3,9]	$27 < 38 \rightarrow \text{true}$; best index becomes 1; $43 < 27 \rightarrow \text{false}$; best stays 1; $3 < 27 \rightarrow \text{true}$; best index becomes 3; $9 < 3 \rightarrow \text{false}$; best stays 3; Scan is complete: swap indexes 0 and 3.	[3,27,43,38,9]	4
2	[3,27,43,38,9]	$43 < 27 \rightarrow \text{false}$; best stays 1; $38 < 27 \rightarrow \text{false}$; best stays 1; $9 < 27 \rightarrow \text{true}$; best index becomes 4; Scan is complete: swap indexes 1 and 4.	[3,9,43,38,27]	3
3	[3,9,43,38,27]	$38 < 43 \rightarrow \text{true}$; best index becomes 3; $27 < 38 \rightarrow \text{true}$; best index becomes 4; Scan is complete: swap indexes 2 and 4.	[3,9,27,38,43]	2
4	[3,9,27,38,43]	$43 < 38 \rightarrow \text{false}$; best stays 3; Minimum is already in position; no swap.	[3,9,27,38,43]	1

10 key comparisons; 3 swaps.

5. Descending target

8

3

5

1

0

1

2

3

↓ descending selection sort

8

5

3

1

0

1

2

3

The trace below connects the input diagram to the final result.

Pass	Before	Evaluated conditions	After	Key comparisons
1	[8,3,5,1]	$3 > 8 \rightarrow \text{false}$; best stays 0; $5 > 8 \rightarrow \text{false}$; best stays 0; $1 > 8 \rightarrow \text{false}$; best stays 0; Minimum is already in position; no swap.	[8,3,5,1]	3
2	[8,3,5,1]	$5 > 3 \rightarrow \text{true}$; best index becomes 2; $1 > 5 \rightarrow \text{false}$; best stays 2; Scan is complete: swap indexes 1 and 2.	[8,5,3,1]	2
3	[8,5,3,1]	$1 > 3 \rightarrow \text{false}$; best stays 2; Minimum is already in position; no swap.	[8,5,3,1]	1

6 key comparisons; 1 swaps. Select the maximum instead: change $\text{arr}[j] < \text{arr}[\text{min}]$ to $\text{arr}[j] > \text{arr}[\text{min}]$. The prefix then grows in descending order.

12. Properties and why the complexity occurs

Property	Result	Reason
Best / average / worst	$O(n^2)$	Every suffix is fully scanned, even on sorted input.
Key comparisons	$n(n-1)/2$	Lengths $n-1, n-2, \dots, 1$.
Swaps	At most $n-1$	At most one actual exchange per pass; self-swaps are skipped.
Auxiliary space	$O(1)$ iterative; $O(n)$ recursive	Only index variables and a temporary value in the iterative form.
Stable / in place / adaptive	No / yes / no	A long swap can cross equal records; scanning does not shrink with presortedness.

Pass 1		$n-1$ comparisons
Pass 2		$n-2$ comparisons
Last pass		1 comparison

Worst-case triangular work: $(n-1) + (n-2) + \dots + 1 = n(n-1)/2$.

For $n=5$, the full suffix scans make $4+3+2+1=10$ key comparisons even on sorted input. Fewer swaps do not imply fewer comparisons.

13. When to choose it

Useful when swaps are more costly than comparisons and the input is small. It offers few swaps but still requires quadratic comparisons.

14. Common mistakes, symptoms and repairs

Attempt	What goes wrong	Repair
Set $\text{min} = \text{arr}[i]$	Treats a value as an index.	Set $\text{min} = i$, then compare $\text{arr}[j]$ and $\text{arr}[\text{min}]$.
Swap every new minimum immediately	Changes the taught selection procedure and swap counts.	Finish the full scan, then swap once.
Always swap min and i	Adds redundant writes when they are the same.	Skip the swap when $\text{min} == i$.

Attempt	What goes wrong	Repair
Call it stable	Long-distance exchanges can cross equal values.	Use a tagged duplicate counterexample.

15. Exam-style practice and worked answers

A. Trace [8, 3, 5, 1] in descending order. Show every completed pass/merge. [8]

Pass	Before	After	Key comparisons
1	[8,3,5,1]	[8,3,5,1]	3
2	[8,3,5,1]	[8,5,3,1]	2
3	[8,5,3,1]	[8,5,3,1]	1

Final result [8,5,3,1]. Select the maximum instead: change $\text{arr}[j] < \text{arr}[\text{min}]$ to $\text{arr}[j] > \text{arr}[\text{min}]$. The prefix then grows in descending order.

B. Explain stability with tagged equal keys. [4]

Swap 1 with 2a in [2a, 2b, 1]: result [1, 2b, 2a]. The equal records crossed, so the taught version is not stable.

C. Give the time and space, then justify both. [5]

Best / average / worst: $O(n^2)$. Every suffix is fully scanned, even on sorted input. Key comparisons: $n(n-1)/2$. Lengths $n-1$, $n-2$, ..., 1. Swaps: At most $n-1$. At most one actual exchange per pass; self-swaps are skipped. Auxiliary space: $O(1)$ iterative; $O(n)$ recursive. Only index variables and a temporary value in the iterative form. Stable / in place / adaptive: No / yes / no. A long swap can cross equal records; scanning does not shrink with presortedness.

D. Debug the code before tracing it. [4]

Mistake	Symptom	Fix
Set $\text{min} = \text{arr}[i]$	Treats a value as an index.	Set $\text{min} = i$, then compare $\text{arr}[j]$ and $\text{arr}[\text{min}]$.
Swap every new minimum immediately	Changes the taught selection procedure and swap counts.	Finish the full scan, then swap once.

E. What happens for [] and [7]? [2]

Both are already sorted. No key comparisons or value movements are required. Merge sort returns at its base case; the iterative methods never enter a pass.

16. Test-ready summary and mastery check

Scan the entire unsorted suffix, remember its minimum index, and swap that minimum into the next final position.

- I can introduce every variable before using it.
- I can draw the current array with index labels.
- I can evaluate conditions with the actual values.
- I can distinguish pass summaries from individual operations.
- I can trace descending and duplicate cases.
- I can justify stability, time and auxiliary space.
- I can test empty, singleton, sorted and reverse input.

CSI247 visual study guide · Selection Sort · 2026-10-07.1

Structural reference: the author-supplied CSI247 guide, with topic-specific adaptation and repo-owned teaching content. All traces in this artifact are generated by deterministic simulations of its stated procedures. Counts distinguish key comparisons from loop tests, probes, swaps, shifts and array writes.